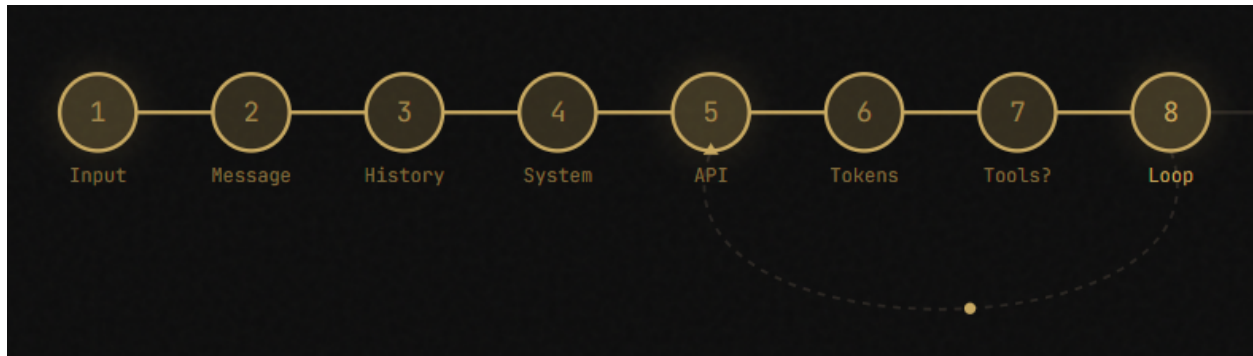
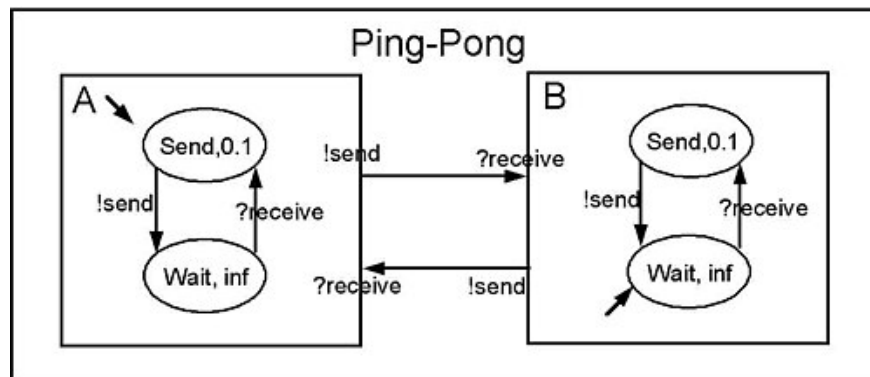


Week 6 : States, Route, Manage



<https://ccunpacked.dev/>

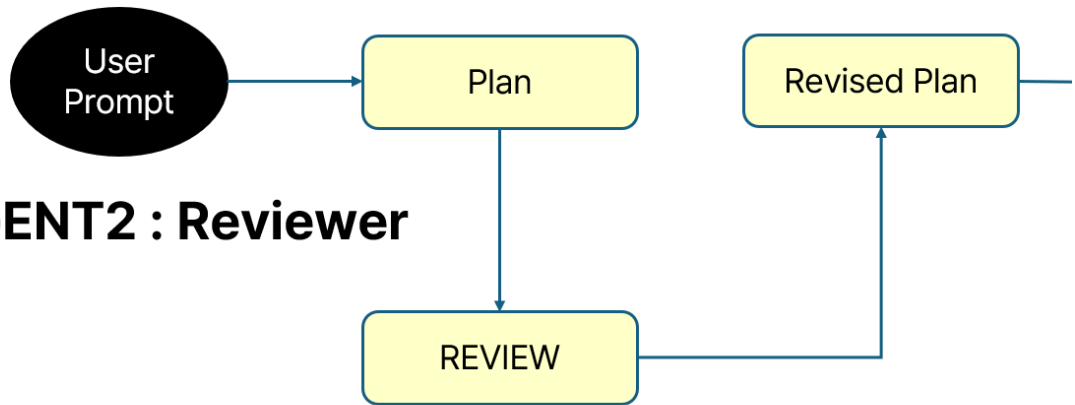
Ping-Pong Game



에이전틱 코딩 워크플로우/세션에서 각 에이전트는:

- 상태(Processing | Idle)와 상태 변화(전이)를 가진다
- 입력과 출력을 가진다
- 시간의 개념을 가지게 된다

- **AGENT1 : Planner**

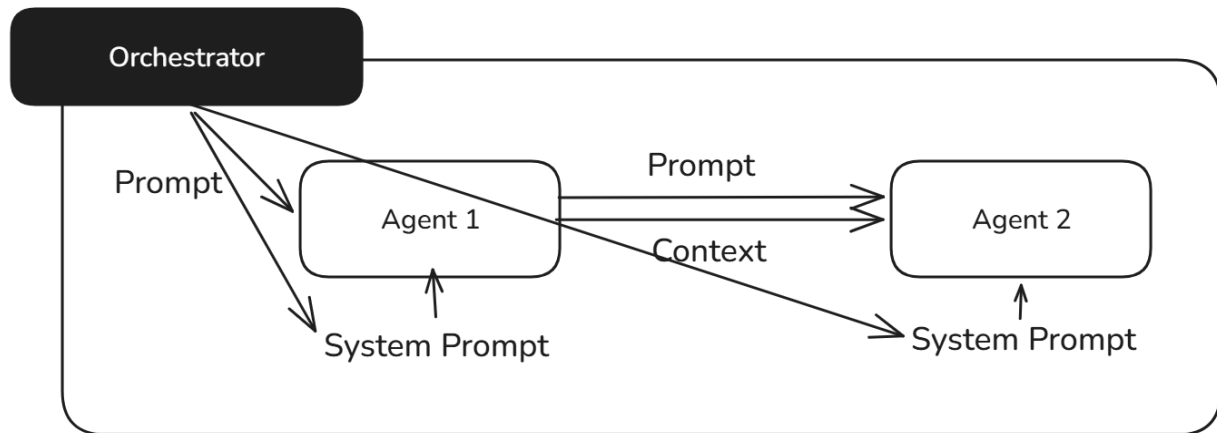


- **AGENT2 : Reviewer**

지난 시간의 다이어그램에서:

- Planner가 Reviewer에게 자료를 전달 후 대기
- Reviewer는 Planner의 요청에 따라 작업 후 Review 반환 → 이후 Sleep (또는 Terminate)

In Workflow

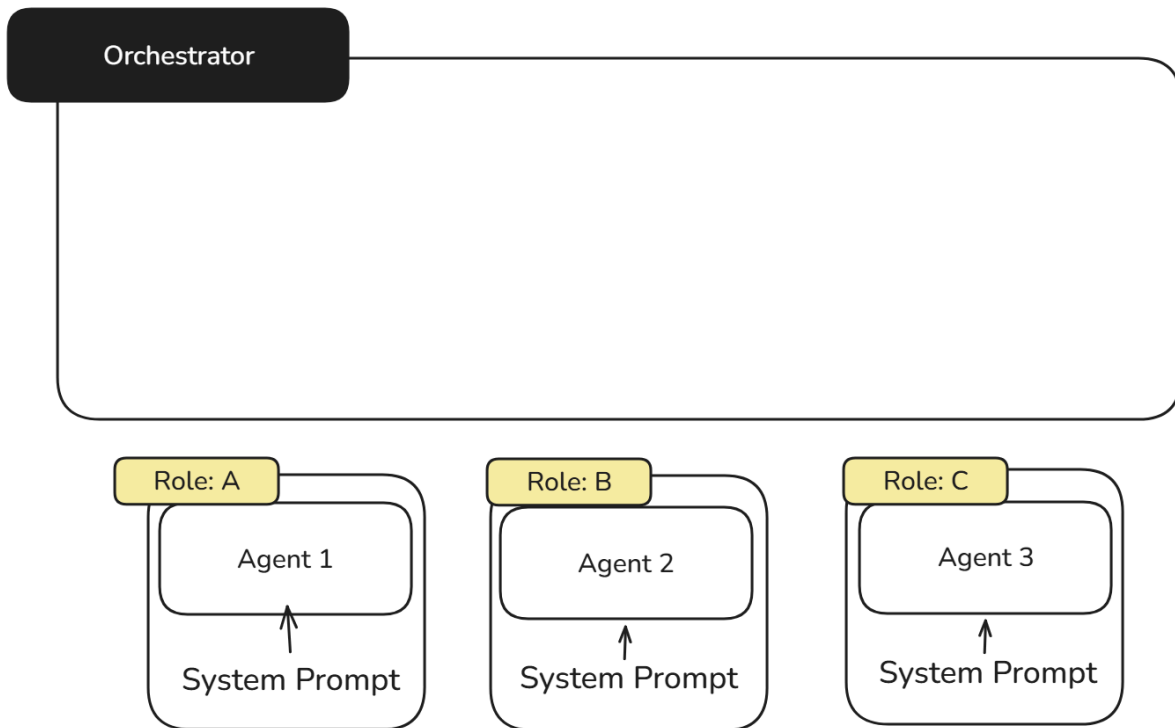


Limitations of Workflow (Pipeline)

- Main Orchestrator에게 과도한 부담 (Sub-Agent 방식을 그대로 재사용)

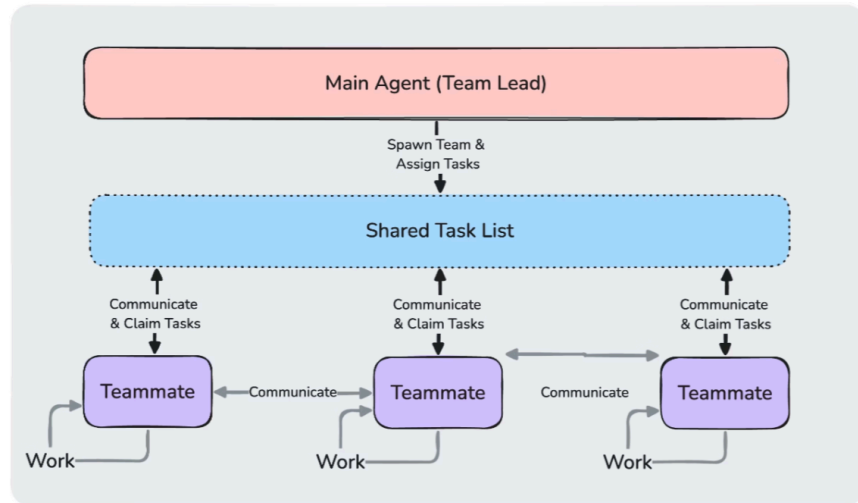
- 단일 작업에 활용, 파이프라인 자체를 Tool처럼 보관하는게 아니라면 이후 재사용 어려움

Agent Pool



- 개별 에이전트의 설정값을 XML, TOML, JSON 등으로 관리
- 필요한 에이전트를 Pool에 등록하여 Orchestrator가 유동적으로 활용 가능하도록 처리

Agent Teams



- 강의시간에는 단순하게 CLI Tool을 통해서 진행하지만, Subprocess를 활용하면 Agent Team을 구축 할 수도 있음

```

{
  "id": "에이전트 파일명과 동일",
  "role": "에이전트의 한 줄 역할 설명",
  "system_prompt": "에이전트에게 전달될 전체 시스템 프롬프트",
  "input": {
    "description": "이 에이전트가 받는 입력 설명",
    "format": "입력 형식 (text, csv, json, file_path 등)",
    "source": "입력이 어디서 오는지 (user, 이전 에이전트 ID, file 등)"
  },
  "output": {
    "description": "이 에이전트가 만드는 출력 설명",
    "format": "출력 형식",
    "files": ["생성하는 파일 경로 목록"]
  },
  "tools": ["이 에이전트가 사용할 수 있는 도구/명령 목록"],
  "constraints": {
    "max_attempts": 3,
    "timeout_seconds": 180,
    "sandbox": "이 에이전트의 권한 수준"
  }
}
  
```

```
}
}
```

Codex_CLI_Subprocess_API.md 의 subprocess 호출 방식을 참고하여 [Pasted text #1 +20 lines] 제안된 구조의방식으로 (1) 계획을 수립하는 Plan Agent, (2) 계획을 평가하는 Review Agent 를 JSON 양식으로 정의해주시겠어요 ? /pool 디렉토리로 관리하면 좋을 것 같아요. 모든 에이전트는 Subprocess 방식으로 호출됩니다.

- 참고: Codex CLI 호출시에는 프로젝트가 Sandbox로 감싸져있어야 합니다. Codex 에게 현재 작업 디렉토리를 git init 해달라고 해주세요.

```
"id": "review_agent",
"role": "Plan Agent의 계획을 안전성, 완전성, 실현 가능성 관점에서 평가하는 에이전트",
"system_prompt": "너는 소프트웨어 엔지니어링 계획의 리뷰 전문가이자 안전 게이트다.\n\n## 역할\nPlan Agent가 수
"input": {
  "description": "Plan Agent가 생성한 계획 JSON",
  "format": "json",
  "source": "plan_agent"
},
```

.pool/ 디렉토리의 에이전트들을 테이블 형태로 보여주는 대시보드를 만들어주세요.

- 에이전트가 추가/삭제되면 테이블에 동적으로 반영
- 각 에이전트 행에 현재 상태(idle/running/completed/failed)를 실시간 표시
- 에이전트가 실행될 때마다 라운드 번호와 해당 출력을 이력으로 누적
- 파이프라인 순서는 고정하지 말고, 어떤 에이전트든 풀에 등록되면 테이블에 나타나게

AP Agent Pool 2 agents Connected

POOL TABLE

STATUS	AGENT ID	ROLE	SANDBOX	ROUNDS	LATEST OUTPUT	ACTIONS
	plan_agent	사용자 요청을 분석하여 실행 가능한 단계별 계획을 수립하는 에이전트	read-only	0	-	View Rounds
	review_agent	Plan Agent의 계획을 안전성, 완전성, 실현 가능성 관점에서 평가하는 에이전트	read-only	0	-	View Rounds

- 샘플입니다.

review agent가 현재 코드를 평가하여 문서화 하고, plan 에이전트가 리팩터링 계획을 짜도록 한 뒤, coder 에이전트와 test agent를 비슷한 방법으로 생성 후 풀에 등록합니다.

STATUS	AGENT ID	ROLE	SANDBOX	ROUNDS	LATEST OUTPUT	ACTIONS
●	coder_agent	Plan Agent의 계획을 실제 코드로 구현하는 에이전트	workspace-write	0	-	View Rounds
●	plan_agent	사용자 요청을 분석하여 실행 가능한 단계별 계획을 수립하는 에이전트	read-only	0	-	View Rounds
●	review_agent	Plan Agent의 계획을 안전성, 완전성, 실현 가능성 관점에서 평가하는 에이전트	read-only	1	전체적으로 잘 구축된 에이전...	View Rounds
●	test_agent	Coder Agent의 구현 결과를 검증하는 테스트 에이전트	read-only	0	-	View Rounds

Rule of Orchestrator

- Subprocess를 이용하여 에이전트의 행동을 제한할 수 있도록 컨텍스트 파일 (CLAUDE.md, AGENTS.md, GEMINI.md)를 만듭니다.

당신은 Orchestrator이기 때문에 pool/*.json의 에이전트 정의를 읽고, 목표에 맞는 에이전트를 선택하여 실행해야 합니다. 에이전트 결과는 반드시 독립 subprocess를 통해 실행되고, 생성되어야 합니다. 이에 따라, 현재 디렉토리 최상위에 컨텍스트 파일을 생성해주시오.

Task

- CLI 에이전트에게 웹을 통해 GitHub 트렌드를 조사하고 리포트를 작성하는 자동화 Task를 수행하려면 어떤 에이전트가 필요할지 논의하고 구현합니다.
- 추가된 에이전트들이 에이전트 풀에 등록되었는지 확인하고, 실제 진행을 대시보드에서 추적합니다.